МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 4
«Дослідження та реалізація криптографічних механізмів»
з дисципліни «Безпека програмного забезпечення»

Варіант 14

Виконав:
здобувач групи ІП-34
Скачков І. О.

Перевірив:
Курченко О. А.

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи — набути практичних навичок застосування криптографічних примітивів для забезпечення конфіденційності, цілісності та автентичності даних і навчитися розпізнавати типові помилки їх використання у програмному коді.

Завдання роботи:

1) виконати покрокову верифікацію навчальної реалізації RSA на малих числах;
2) реалізувати автентифіковане шифрування за схемою AEAD варіанта з унікальним nonce і прив'язкою асоційованих даних;
3) експериментально продемонструвати наслідки двох типових помилок варіанта;
4) сформувати та перевірити електронний підпис за схемою варіанта;
5) обґрунтувати висновки про криптостійкість і межі застосовності навчального RSA.

Параметри варіанта отримано командою `python -m common.variants 14 --lab 4`:

```
  Варіант 14

    LAB4
      aead: AES-128-GCM
      signature: RSA-PSS (2048)
      rsa_demo_bits: 28
      misuse_to_demonstrate:
        - режим ECB замість AEAD
        - хешування пароля без солі
```

Таблиця 1.1 – Індивідуальне завдання (варіант 14)

Параметр	Значення
Схема AEAD	AES-128-GCM
Схема підпису	RSA-PSS (2048 біт)
Розрядність множників демонстраційного RSA	28 біт
Помилка 1	режим ECB замість AEAD
Помилка 2	хешування пароля без солі

2 ТЕОРЕТИЧНЕ ОБҐРУНТУВАННЯ ПРИЙНЯТИХ РІШЕНЬ

2.1 AES-128-GCM: nonce і асоційовані дані

Стійкість GCM тримається на унікальності nonce за фіксованого ключа. Обрано 96-бітовий nonce із криптографічного ГВЧ ОС (os.urandom) для кожного повідомлення: це рекомендований для GCM розмір, за якого лічильник не потребує додаткового обчислення через GHASH. Випадковий nonce не потребує збереження стану між запусками; ціна — обмеження на кількість повідомлень одним ключем: NIST SP 800-38D допускає не більше 2^{32} шифрувань із випадковими nonce, щоб імовірність повторення лишалася нехтовно малою. Ліміт реалізовано класом NonceBudget, що вимагає ротації ключа. Асоційовані дані передаються в AESGCM.encrypt/decrypt і входять у тег, тож заголовок неможливо змінити або «перенести» на інший шифротекст. Довжина ключа перевіряється явно: клас AESGCM обирає варіант AES за довжиною ключа, і 32-байтовий ключ мовчки дав би AES-256 замість схеми варіанта.

2.2 Демонстрація помилок

Для ECB обрано три докази: повторювані 16-байтові блоки тексту, растрове зображення з однотонними ділянками (найнаочніший приклад того, що ECB не приховує структуру) та атака перестановкою блоків, яка показує втрату цілісності. Для хешування без солі змодельовано злиту базу з чотирьох облікових записів і заздалегідь обчислену таблицю поширених паролів; виправлений варіант використовує scrypt зі стандартної бібліотеки ($N = 2^{14}$, $r = 8$, $p = 1$, 16-байтова сіль).

2.3 RSA-PSS

Підпис RSA формується над SHA-256 з доповненням PSS (MGF1-SHA256, сіль максимальної довжини). PSS імовірнісний: повторний підпис того самого повідомлення дає інший рядок, а схема має доведену стійкість на відміну від детермінованого PKCS#1 v1.5. Для модуля 2048 біт максимальна довжина солі за RFC 8017 — emLen − hLen − 2 = 256 − 32 − 2 = 222 байти.

3 ОПИС РОЗРОБЛЕНОГО РІШЕННЯ

3.1 Структура та програмні засоби

```
SoS/lab4/lab04_crypto/
  crypto_lab/rsa_edu.py        навчальний RSA (каркас курсу, без змін)
  crypto_lab/aead.py           AES-128-GCM: encrypt, decrypt, NonceBudget
  crypto_lab/misuse_ecb.py     помилка 1: ECB, cut-and-paste, зображення
  crypto_lab/misuse_hash.py    помилка 2: SHA-256 без солі проти scrypt
  crypto_lab/sign_demo.py      RSA-PSS 2048: генерація, підпис, перевірка
  crypto_lab/rsa_analysis.py   факторизація, кількість множень
  demo.py                      усі кроки роботи, рисунки
  tests/test_variant.py        24 тестових функцій (26 тестів)
```

Використано Python 3.12, cryptography 43.0.1 і pytest 8.3.3 (версії з requirements.txt каркаса), matplotlib 3.9.2 для рисунків. Факторизацію ρ-методом Полларда й розширений алгоритм Евкліда з протоколом кроків написано самостійно.

3.2 Автентифіковане шифрування (aead.py)

Результат encrypt — nonce (12 байтів) ‖ шифротекст ‖ тег (16 байтів). decrypt відокремлює nonce й передає решту в AESGCM.decrypt, який перевіряє тег до повернення відкритого тексту; за розбіжності — виняток InvalidTag, дані не повертаються:

```
def encrypt(plaintext: bytes, key: bytes, associated: bytes = b"") -> bytes:
    """БЕЗПЕЧНО: новий випадковий nonce, асоційовані дані входять у тег."""
    _check_key(key)
    nonce = os.urandom(NONCE_SIZE)
    return nonce + AESGCM(key).encrypt(nonce, plaintext, associated)



def decrypt(blob: bytes, key: bytes, associated: bytes = b"") -> bytes:
    """Відокремлює nonce і перевіряє тег; за розбіжності — InvalidTag."""
    _check_key(key)
    if len(blob) < NONCE_SIZE + TAG_SIZE:
        raise InvalidTag()
    nonce, body = blob[:NONCE_SIZE], blob[NONCE_SIZE:]
    return AESGCM(key).decrypt(nonce, body, associated)
```

3.3 Помилка 1: ECB

```
# ВРАЗЛИВО: ECB — детермінований, без nonce і без контролю цілісності.
def ecb_encrypt(data: bytes, key: bytes) -> bytes:
    padder = padding.PKCS7(8 * BLOCK).padder()
    padded = padder.update(data) + padder.finalize()
    enc = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
    return enc.update(padded) + enc.finalize()
```

Атака перестановкою блоків: порушник замінює блок ролі у своєму шифротексті блоком із шифротексту іншого користувача, зашифрованого тим самим ключем:

```
def cut_and_paste(victim_ct: bytes, donor_ct: bytes) -> bytes:
    """Замінити блок ролі шифротексту жертви блоком із чужого шифротексту."""
    return victim_ct[:BLOCK] + donor_ct[BLOCK:2 * BLOCK] + victim_ct[2 * BLOCK:]
```

3.4 Помилка 2: хешування без солі

```
# ВРАЗЛИВО: швидкий хеш без солі — однакові паролі → однакові хеші.
def hash_unsalted(password: str) -> str:
    return hashlib.sha256(password.encode()).hexdigest()


# БЕЗПЕЧНО: унікальна сіль + повільна KDF; формат scrypt$N$r$p$сіль$ключ.
def hash_salted(password: str) -> str:
    salt = os.urandom(SALT_SIZE)
    dk = hashlib.scrypt(password.encode(), salt=salt,
                        n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P)
    s, k = base64.b64encode(salt).decode(), base64.b64encode(dk).decode()
    return f"scrypt${SCRYPT_N}${SCRYPT_R}${SCRYPT_P}${s}${k}"
```

3.5 Підпис RSA-PSS (sign_demo.py)

```
PSS = padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                  salt_length=padding.PSS.MAX_LENGTH)
HASH = hashes.SHA256()



def generate_keypair() -> rsa.RSAPrivateKey:
    # ТЕСТОВИЙ КЛЮЧ — не використовувати поза стендом
    return rsa.generate_private_key(public_exponent=PUBLIC_EXPONENT,
                                    key_size=KEY_BITS)



def sign(priv: rsa.RSAPrivateKey, message: bytes) -> bytes:
    return priv.sign(message, PSS, HASH)



def verify(pub: rsa.RSAPublicKey, signature: bytes, message: bytes) -> bool:
    """True — підпис дійсний; False — дані або підпис змінено."""
    try:
        pub.verify(signature, message, PSS, HASH)
        return True
    except InvalidSignature:
        return False
```

3.6 Аналіз навчального RSA

rsa_analysis.py рахує множення fast_pow_mod за протоколом кроків (одне піднесення до квадрата на кожен біт показника плюс одне множення на основу на кожен одиничний біт) і факторизує модуль повним перебором та ρ-методом Полларда (варіант Брента):

```
def multiplications(exp: int) -> tuple[int, int]:
    """(квадратів, множень на основу) у fast_pow_mod для показника exp."""
    _, steps = fast_pow_mod(3, exp, 1_000_003)
    return len(steps), bin(exp).count("1")
```

4 РЕЗУЛЬТАТИ

4.1 Покрокова верифікація RSA

Протокол fast_pow_mod, тест Міллера — Рабіна, розширений алгоритм Евкліда та контрольні приклади (вивід demo.py, розділ 1):

```
fast_pow_mod(7, 13, 100) = 7 (pow: 7); протокол «квадрат і множення»:
    e=1101 bit=1 b=7 result=7
    e= 110 bit=0 b=49 result=7
    e=  11 bit=1 b=1 result=7
    e=   1 bit=1 b=1 result=7
is_probable_prime(7919) = True
is_probable_prime(7917) = False
is_probable_prime(561) = False  (число Кармайкла 561 = 3·11·17)
is_probable_prime(104729) = True
egcd(240, 46) = (2, -9, 47): 240·-9 + 46·47 = 2
Розширений Евклід для 17⁻¹ mod 3120 (q — частка, r — залишок, x — коефіцієнт при 17):
    r =    17  q =  183  x = 1
    r =     9  q =    1  x = -183
    r =     8  q =    1  x = 184
    r =     1  q =    8  x = -367
    1 = 17·(-367) + 3120·2 → d = -367 mod 3120 = 2753
modinv(17, 3120) = 2753
Контрольний приклад: p = 61, q = 53, n = 3233, φ = 3120, e = 17, d = 2753; 65^17 mod 3233
= 2790; 2790^2753 mod 3233 = 65
Контрольне запитання 9: p = 11, q = 23 → n = 253, φ = 220; d = 3⁻¹ mod 220 = 147; e·d =
441 = 2·220 + 1
```

Усі значення збіглися з очікуваними: $7^{13} \bmod 100 = 7$, 7919 — просте, $7917 = 3 \cdot 7 \cdot 13 \cdot 29$ — складене, число Кармайкла 561, яке пропускає тест Ферма, тест Міллера — Рабіна відкидає; $d = 17^{-1} \bmod 3120 = 2753$, $65^{17} \bmod 3233 = 2790$. Для контрольного запитання 9: $d = 3^{-1} \bmod 220 = 147$, бо $3 \cdot 147 = 441 = 2 \cdot 220 + 1$. Еталонні тести tests/test_rsa.py — 6 passed.

4.2 Демонстраційні ключі (28 біт)

Таблиця 4.1 – Згенеровані демонстраційні ключі RSA

Величина	Значення
p (28 біт, просте)	153715963
q (28 біт, просте)	210726973
$n = p \cdot q$	32392099584769999 (55 біт)
$\varphi(n) = (p-1)(q-1)$	32392099220327064
e	65537
$d = e^{-1} \bmod \varphi(n)$	5560387509783473
$e \cdot d \bmod \varphi(n)$	1

```
fast_pow_mod для e (17 біт): квадратів 17, множень на основу 2, разом 19 (наївно: 65536)
fast_pow_mod для d (53 біт): квадратів 53, множень на основу 24, разом 77 (наївно:
5560387509783472)
Атака: ρ-метод Полларда розклав n за 3.19 мс: 153715963 · 210726973; відновлений d =
5560387509783473 (збігається: True)
```

Шифрування m = 42 і розшифрування повернули 42. Для e = 65537 швидке піднесення виконало 19 множень (17 квадратів + 2 множення на основу) замість 65 536, для 53-бітового d — 77 замість $\approx 5{,}6 \cdot 10^{15}$. ρ-метод Полларда розклав модуль за 3,19 мс і відновив закритий ключ d.

4.3 Час генерації простих і факторизації

Таблиця 4.2 – Час генерації простого числа й факторизації n (медіана з 3 запусків)

Біт p, q	Біт n	gen_prime, мс	Повний перебір, мс	ρ Полларда, мс
12	24	0,029	0,08	0,03
16	32	0,044	1,32	0,12
20	40	0,061	29,08	0,36
24	47	0,060	335,53	0,76
28	56	0,049	—	3,26
32	64	0,123	—	22,47
36	72	0,131	—	45,33
40	80	0,157	—	403,58
44	88	0,223	—	394,27

Генерація 512- і 1024-бітового простого числа (для RSA-1024 і RSA-2048) тривала 60 і 64 мс. Залежність часу факторизації від розрядності — на рис. 4.1.

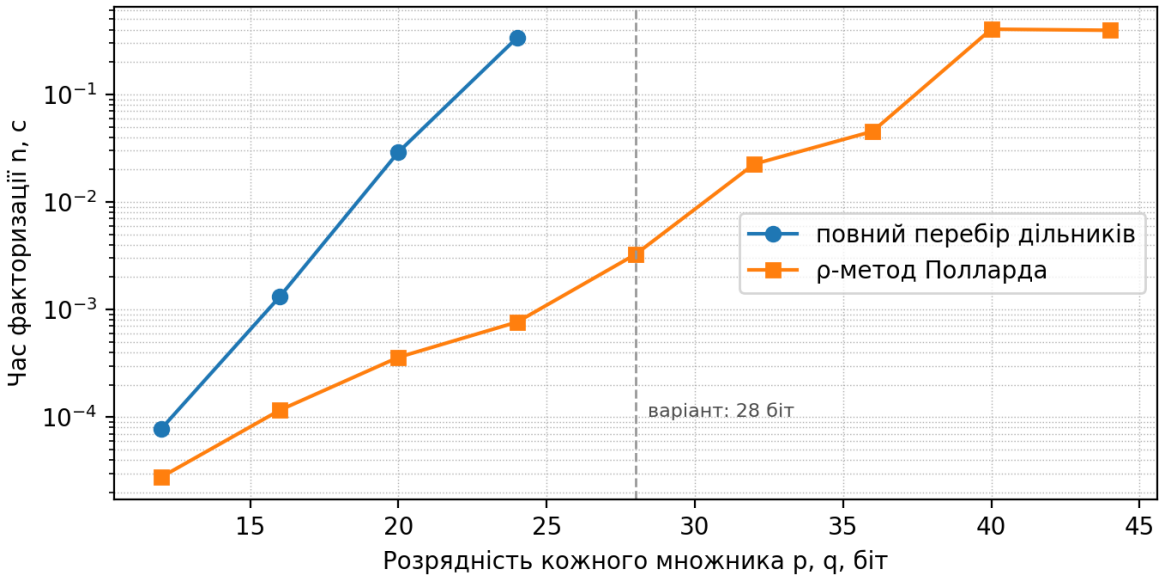


Рисунок 4.1 – Час факторизації модуля залежно від розрядності множників

4.4 AES-128-GCM

```
ключ 16 Б; blob = nonce 12 + шифротекст 14 + тег 16 = 42 Б
розшифрування: b'secret payload'
  змінено 1 біт тегу                    InvalidTag — дані не повернено
  змінено 1 біт шифротексту             InvalidTag — дані не повернено
  змінено асоційовані дані (hdr → hdR)  InvalidTag — дані не повернено
  змінено nonce                         InvalidTag — дані не повернено
  інший ключ                            InvalidTag — дані не повернено
10 000 шифрувань: унікальних nonce 10000
той самий текст двічі → однакові blob: False
```

Зміна одного біта тегу, шифротексту чи nonce, зміна асоційованих даних або ключа дають InvalidTag; за 10 000 шифрувань повторних nonce немає; той самий відкритий текст щоразу дає інший шифротекст.

4.5 Помилка 1: ECB замість AEAD

```
відкритий текст: 6 блоків, з них повторів: 4
ECB, блоки шифротексту:
    0: 6847531d5ec6ebf44a3549a6e2d32075
    1: 6847531d5ec6ebf44a3549a6e2d32075
    2: 6847531d5ec6ebf44a3549a6e2d32075
    3: 6847531d5ec6ebf44a3549a6e2d32075
    4: d13c2c46e4c24fc748b8083da17faf39
    5: 6847531d5ec6ebf44a3549a6e2d32075
    6: 377222e061a924c591cd9c27ea163ed4
ECB: повторів блоків у шифротексті 4; GCM: 0
Атака перестановкою блоків (cut-and-paste):
    ECB розшифровує підробку без помилки: b'user=mallory....role=admin......'
    GCM: InvalidTag — підробку відхилено
зображення 256×256 (65536 байтів): повторів блоків — відкрите 4051, ECB 4051, GCM 0
```

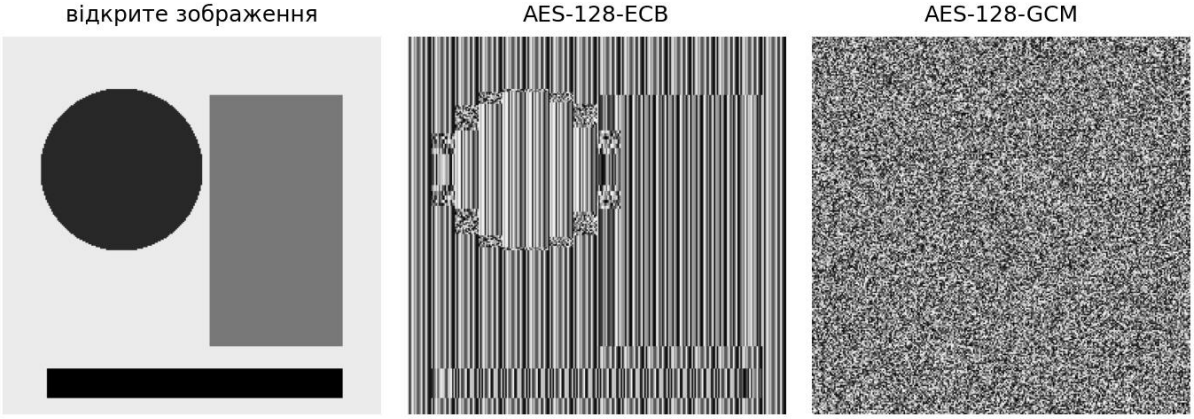


Рисунок 4.2 – Синтетичне зображення, зашифроване AES-128 в режимах ECB і GCM

У зображенні 256 × 256 4051 з 4096 блоків повторюють інші; в ECB-шифротексті повторів рівно стільки ж (4051), тому контури фігур видно (рис. 4.2), у GCM — 0.

4.6 Помилка 2: хешування пароля без солі

```
  ВРАЗЛИВО alice  daaad6e5604e8e17bd9f108d91e26afe6281dac8fda0091040a7a6d7bd9b43b5
  ВРАЗЛИВО bob    daaad6e5604e8e17bd9f108d91e26afe6281dac8fda0091040a7a6d7bd9b43b5
  ВРАЗЛИВО carol  48486e1514e842346ff405b1e45f44059ae82619f2306f99d0940dcb386e91f7
  ВРАЗЛИВО dave   0b14d501a594442a01c6859541bcb3e8164d183d32937b851835442f69d5c94e
  alice і bob мають однаковий хеш: True
  таблиця з 15 хешів (будується заздалегідь, 0.02 мс); пошук: 10.6 мкс → зламано 3 з 4:
{'alice': 'qwerty123', 'bob': 'qwerty123', 'dave': 'password1'}
  БЕЗПЕЧНО alice  scrypt$16384$8$1$o4azEmfaA2GbBwq7wMT1Ug==$DD8OwZDr1+jEvK8jtz…
  БЕЗПЕЧНО bob    scrypt$16384$8$1$bQnAP8zWlmX4C0oTFFBEYA==$SgNvpDr6rgwUosvFrr…
  БЕЗПЕЧНО carol  scrypt$16384$8$1$ne7uheFkSTwSBdLyxv04UA==$XANQpvoGZ7SxWwBiUg…
  БЕЗПЕЧНО dave   scrypt$16384$8$1$bw9Rw8DG6bhEnq2de06LKQ==$CrJIvmLqRp9HL0yrKO…
  alice і bob мають однаковий запис: False
  зламано тією самою таблицею: 0 з 4
  перебір словника для одного запису (dave) зі scrypt: знайдено 'password1' за 0.50 с (≈
62 мс на кандидата)
  SHA-256: 0.83 мкс на кандидата
```

4.7 Підпис RSA-PSS

```
Ключ RSA: 2048 біт, e = 65537; генерація 11 мс
Повідомлення: b'lab4 demo message: order #2026-14 approved'
Підпис: 256 Б, сіль PSS 222 Б; підпис 1.34 мс, перевірка 0.07 мс
Підпис (перші 32 байти):
1c13e41569d405911a5f23f1f004aa5ffb11296bfe106c6419610cca0b36a25d…
Перевірка коректного підпису:         True
Повідомлення + b'x':                  False
Змінено 1 біт повідомлення:           False
Змінено 1 біт підпису:                False
Повторний підпис того самого тексту:  інший | дійсний: True
Перевірка чужим відкритим ключем:     False
Відкритий ключ (PEM, 451 Б): MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAo+sx…
```

4.8 Додатково: повторний nonce у GCM

Для контрольного запитання 3 два повідомлення зашифровано тим самим ключем і nonce:

```
C1 ⊕ C2 == P1 ⊕ P2: True
знаючи P1, порушник відновлює P2: b'pay 999 UAH to mallo'
гама (keystream) однакова: True
```

4.9 Тестування

Власний файл tests/test_variant.py містить 24 тестових функцій (26 тестів із параметризацією): навчальний RSA (7), AES-128-GCM (7), помилка ECB (4), хеш без солі (3), RSA-PSS (3). Разом з еталонними tests/test_rsa.py — 32 passed; повний звіт — у додатку Б.

5 АНАЛІЗ РЕЗУЛЬТАТІВ

5.1 Генерація простих і факторизація

Час генерації простого числа зростає поліноміально: від сотих часток мілісекунди для 12–44 біт до 64 мс для 1024 біт — тест Міллера — Рабіна виконує модульне піднесення до степеня ($O(k^3)$ бітових операцій), а густина простих спадає лише як $1/\ln n$. Факторизація зростає експоненційно (рис. 4.1): повний перебір множиться приблизно на 16 при +4 бітах множника ($\approx 2^k$), ρ-метод — приблизно на 4 ($\approx 2^{(k/2)}$). Модуль варіанта з 28-бітовими множниками ρ-метод розкладає за 3,26 мс; повний перебір за екстраполяцією — за 5,4 с, а на папері навіть 16-бітові множники потребують ~2^{15} ділень. Екстраполюючи ρ-метод від 0,39 с для 44 біт, отримуємо ≈ 7 хв для 64-бітових множників і ≈ 3 років для 100-бітових — тобто навчальний RSA стає непрактичним для факторизації на ПК десь від 80–100 біт на множник. Модуль 2048 біт (множники по 1024 біт) ρ-методу недосяжний ($\approx 2^{512}$ кроків); найкращий відомий алгоритм — решето числового поля — потребував близько 2700 ядро-років для 829-бітового RSA-250 (2020 р.), а 2048-бітовий модуль за NIST SP 800-57 відповідає стійкості 112 біт.

5.2 Кількість множень

fast_pow_mod виконує одне піднесення до квадрата на кожен біт показника і одне множення на кожен одиничний біт, тобто від k до 2k множень для k-бітового показника. Для e = 65537 (17 біт, два одиничні біти) — 19 множень; для 53-бітового d — 77. Наївне піднесення потребує exp − 1 множень: 65 536 для e і $\approx 5{,}6 \cdot 10^{15}$ для d, а для 2048-бітового показника — $\approx 2^{2048}$, що фізично нездійсненне. Саме логарифмічна складність робить RSA практичним для легітимних сторін.

5.3 Що показують демонстрації помилок

ECB втрачає дві властивості. Конфіденційність структури: пасивний порушник, який лише бачить шифротекст, знаходить повтори (4 однакові блоки з 6 у тексті, контури фігур на рис. 4.2) і може судити про зміст без ключа. Цілісність: активний порушник, що має шифротексти інших записів того самого ключа,

переставляє блоки — підроблений запис «user=mallory, role=admin» розшифровується без жодної помилки, тоді як GCM ту саму підробку відхиляє. **Хеш без солі** втрачає конфіденційність паролів після витоку бази: порушник із готовою таблицею поширених паролів зламав 3 з 4 записів за 10,6 мкс, без жодного обчислення хешу, і бачить, що в alice і bob однаковий пароль. З сіллю й scrypt таблиця не зламала жодного запису, а перебір словника для одного запису коштує 62 мс на кандидата замість 0,83 мкс.

5.4 AEAD проти шифрування без контролю цілісності

Шифрування без MAC (ECB у роботі) приховує лише байти, але не захищає від зміни: розшифрування підробленого шифротексту завершується успішно, і застосунок отримує дані, яких ніхто не відправляв (розділ 4.5). AEAD перевіряє тег до видачі тексту: будь-яка зміна шифротексту, nonce, тегу або асоційованих даних дає InvalidTag (розділ 4.4), а перестановка блоків між повідомленнями — так само (розділ 4.5). Тобто AEAD захищає від активного порушника, а «чисте» шифрування — лише від пасивного, і то за умови рандомізації.

5.5 Реакція на підміну: тег і підпис

Обидва механізми реагують на зміну одного біта однаково — відмовою (InvalidTag / InvalidSignature). Відмінність — у гарантіях. Тег GCM обчислюється спільним симетричним ключем: його може створити будь-яка сторона, що знає ключ, тому MAC доводить цілісність і походження «від когось із власників ключа», але не дає неспростовності. Підпис RSA-PSS створює лише власник закритого ключа, а перевіряє будь-хто за відкритим: перевірка чужим ключем дала False. Тому підпис дає неспростовність і придатний, коли перевіряльників багато або вони не мають бути здатні підробити дані. Ціна — розмір і час: тег 16 байтів і мікросекунди, підпис 256 байтів, 1,34 мс на підпис і 0,07 мс на перевірку, генерація ключа — 11 мс.

6 ВИСНОВКИ

1) Навчальний RSA верифіковано на контрольних прикладах ($7^{13} \bmod 100 = 7$; d = 2753 і c = 2790 для p = 61, q = 53; d = 147 для p = 11, q = 23); для 28-бітових множників отримано 55-бітовий модуль, $e \cdot d \bmod \varphi(n) = 1$, і ρ-метод Полларда розклав його за 3,19 мс з відновленням d — такий модуль не захищає нічого.
2) Час факторизації зростає експоненційно (≈ 2^(k/2) для ρ-методу), час генерації простих — поліноміально (64 мс для 1024 біт); fast_pow_mod виконує 19 множень для e = 65537 замість 65 536.
3) Реалізовано AES-128-GCM з випадковим 96-бітовим nonce, прив'язкою асоційованих даних і лімітом 2^{32} повідомлень на ключ; зміна будь-якого біта шифротексту, тегу, nonce або асоційованих даних відхиляється з InvalidTag.
4) ECB зберіг усі 4051 повторюваних блоків зображення й пропустив підроблений запис з role=admin; GCM — 0 повторів і відхилена підробка. Хеш без солі дав однакові хеші для однакових паролів і злам 3 з 4 записів таблицею за мікросекунди; сіль і scrypt звели ефективність таблиці до нуля.
5) Підпис RSA-PSS 2048 (256 байтів, сіль 222 байти) перевіряється для коректного повідомлення й відхиляється за зміни одного біта повідомлення, підпису або ключа; повторний підпис рандомізований. Власних тестів 26, разом з еталонними — 32 passed.

7 ВІДОМОСТІ ПРО ВИКОРИСТАННЯ ГЕНЕРАТИВНИХ ІНСТРУМЕНТІВ

Під час виконання роботи як допоміжний засіб використано генеративну модель Claude (Anthropic): для підготовки чернетки коду aead.py, демонстрацій помилок, скрипта підпису, аналізу RSA, тестів і оформлення звіту. Результати перевірено запуском тестів і демонстрації; рішення здобувач перевірив і може пояснити під час захисту.

ПЕРЕЛІК ПОСИЛАНЬ

1) Ferguson N., Schneier B., Kohno T. Cryptography Engineering: Design Principles and Practical Applications. – Indianapolis : Wiley, 2010. – 384 p.
2) Katz J., Lindell Y. Introduction to Modern Cryptography. – 3rd ed. – Boca Raton : CRC Press, 2020. – 648 p.
3) Rivest R. L., Shamir A., Adleman L. A Method for Obtaining Digital Signatures and Public-Key Cryptosystems // Communications of the ACM. – 1978. – Vol. 21, № 2. – P. 120–126.
4) NIST SP 800-38D. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC / M. Dworkin. – Gaithersburg : NIST, 2007.
5) RFC 8017. PKCS #1: RSA Cryptography Specifications Version 2.2 / K. Moriarty [et al.]. – IETF, 2016.
6) NIST SP 800-57 Part 1 Rev. 5. Recommendation for Key Management: Part 1 – General. – Gaithersburg : NIST, 2020.
7) Pollard J. M. A Monte Carlo method for factorization // BIT Numerical Mathematics. – 1975. – Vol. 15, № 3. – P. 331–334.
8) Безпека програмного забезпечення. Частина 2. Лабораторний практикум / В. В. Соколовський. – Київ : КПІ ім. Ігоря Сікорського, 2026.

ДОДАТОК А

Репозиторій і артефакти роботи

Репозиторій: https://github.com/ip34is/7-sem (папка SoS/lab4)

Коміт, що відповідає зданій роботі:

`c05cc9acef593f0df2313317550f784ddaa4d4cd`

Стан роботи на цьому коміті: https://github.com/ip34is/7-sem/tree/c05cc9acef593f0df2313317550f784ddaa4d4cd/SoS/lab4

Артефакти: lab04_crypto/crypto_lab/aead.py, misuse_ecb.py, misuse_hash.py, sign_demo.py, rsa_analysis.py, demo.py, tests/test_variant.py; у каталозі звіт_лр4 — factor_time.png, ecb_image.png, demo_output.txt, pytest_output.txt. Усі ключі синтетичні й генеруються під час запуску; фіксований ключ демонстрацій ECB позначено «ТЕСТОВИЙ КЛЮЧ».

Відтворення результатів (з папки SoS/lab4/lab04_crypto):

```
python -m pip install -r requirements.txt
PYTHONPATH=. pytest tests/ -q        # 32 passed
python demo.py --figs ../звіт_лр4
python -m crypto_lab.sign_demo
```

ДОДАТОК Б

Звіт pytest

```
test_rsa.py::test_fast_pow_matches_builtin PASSED
test_rsa.py::test_miller_rabin_known_primes PASSED
test_rsa.py::test_gen_prime_is_prime PASSED
test_rsa.py::test_egcd_and_modinv PASSED
test_rsa.py::test_rsa_roundtrip PASSED
test_rsa.py::test_textbook_example PASSED
test_variant.py::test_demo_keys_have_28_bit_prime_factors PASSED
test_variant.py::test_control_question_9 PASSED
test_variant.py::test_textbook_ciphertext_value PASSED
test_variant.py::test_carmichael_number_rejected PASSED
test_variant.py::test_fast_pow_multiplication_count PASSED
test_variant.py::test_small_modulus_is_factored_and_key_recovered PASSED
test_variant.py::test_trial_division_finds_smallest_factor PASSED
test_variant.py::test_aead_roundtrip_and_layout PASSED
test_variant.py::test_aead_requires_128_bit_key PASSED
test_variant.py::test_aead_detects_bit_flip[tag] PASSED
test_variant.py::test_aead_detects_bit_flip[ciphertext] PASSED
test_variant.py::test_aead_detects_bit_flip[nonce] PASSED
test_variant.py::test_aead_binds_associated_data PASSED
test_variant.py::test_aead_wrong_key_and_truncated_blob PASSED
test_variant.py::test_aead_nonce_unique_and_encryption_randomized PASSED
test_variant.py::test_nonce_budget_enforces_limit PASSED
test_variant.py::test_ecb_reveals_repeated_blocks_gcm_does_not PASSED
test_variant.py::test_ecb_is_deterministic PASSED
test_variant.py::test_ecb_cut_and_paste_succeeds_gcm_rejects PASSED
test_variant.py::test_ecb_image_structure_survives PASSED
test_variant.py::test_unsalted_equal_passwords_equal_hashes_and_table_cracks PASSED
test_variant.py::test_salted_hashes_differ_and_table_is_useless PASSED
test_variant.py::test_salted_verify PASSED
test_variant.py::test_signature_valid_and_parameters PASSED
test_variant.py::test_signature_rejects_modified_message_or_signature PASSED
test_variant.py::test_pss_is_randomized_and_bound_to_key PASSED
============================== 32 passed in 0.64s ==============================
```

ДОДАТОК В

Лістинги модулів crypto_lab

```
# crypto_lab/aead.py
"""Симетричне автентифіковане шифрування — ЛР4, варіант 14: AES-128-GCM.


Формат результату encrypt:  nonce (12 байтів) ‖ шифротекст ‖ тег (16 байтів).
- nonce — 96 випадкових бітів з os.urandom для КОЖНОГО повідомлення. За випадкових nonce
  NIST SP 800-38D обмежує кількість повідомлень на один ключ величиною 2^32: так імовірність
  повторення nonce лишається нехтовно малою. Ліміт контролює NonceBudget.
- асоційовані дані (заголовок, ідентифікатор) не шифруються, але входять у тег:
  розшифрування з іншими асоційованими даними завершується винятком InvalidTag.
"""
from __future__ import annotations


import os


from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


KEY_SIZE = 16          # AES-128
NONCE_SIZE = 12        # 96 біт — рекомендований розмір nonce для GCM
TAG_SIZE = 16          # 128-бітовий тег
MAX_MESSAGES = 2 ** 32 # межа для випадкових nonce на один ключ (NIST SP 800-38D)


__all__ = ["encrypt", "decrypt", "generate_key", "InvalidTag", "NonceBudget"]



def generate_key() -> bytes:
    return AESGCM.generate_key(bit_length=8 * KEY_SIZE)



def _check_key(key: bytes) -> None:
    if len(key) != KEY_SIZE:
        raise ValueError(f"AES-128-GCM потребує ключ {KEY_SIZE} байтів, отримано {len(key)}")
```

```
def encrypt(plaintext: bytes, key: bytes, associated: bytes = b"") -> bytes:
    """БЕЗПЕЧНО: новий випадковий nonce, асоційовані дані входять у тег."""
    _check_key(key)
    nonce = os.urandom(NONCE_SIZE)
    return nonce + AESGCM(key).encrypt(nonce, plaintext, associated)



def decrypt(blob: bytes, key: bytes, associated: bytes = b"") -> bytes:
    """Відокремлює nonce і перевіряє тег; за розбіжності — InvalidTag."""
    _check_key(key)
    if len(blob) < NONCE_SIZE + TAG_SIZE:
        raise InvalidTag()
    nonce, body = blob[:NONCE_SIZE], blob[NONCE_SIZE:]
    return AESGCM(key).decrypt(nonce, body, associated)



class NonceBudget:
    """Шифратор з лічильником повідомлень: не дає перевищити MAX_MESSAGES на один ключ."""


    def __init__(self, key: bytes, limit: int = MAX_MESSAGES) -> None:
        _check_key(key)
        self.key, self.limit, self.used = key, limit, 0


    def encrypt(self, plaintext: bytes, associated: bytes = b"") -> bytes:
        if self.used >= self.limit:
            raise RuntimeError("вичерпано ліміт повідомлень для ключа — потрібна ротація ключа")
        self.used += 1
        return encrypt(plaintext, self.key, associated)
# crypto_lab/misuse_ecb.py
"""Помилка 1 варіанта 14: режим ECB замість AEAD.


ECB шифрує кожен 16-байтовий блок незалежно тим самим ключем, тому:
  1) однакові блоки відкритого тексту дають однакові блоки шифротексту — структура проступає;
  2) цілісності немає: блоки можна переставляти й підставляти з інших шифротекстів
     того самого ключа, і розшифрування пройде без помилки.
AES-128-GCM (crypto_lab.aead) усуває обидва недоліки.
"""
```

```
from __future__ import annotations

from collections import Counter

from cryptography.hazmat.primitives import padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

BLOCK = 16



# ВРАЗЛИВО: ECB — детермінований, без nonce і без контролю цілісності.
def ecb_encrypt(data: bytes, key: bytes) -> bytes:
    padder = padding.PKCS7(8 * BLOCK).padder()
    padded = padder.update(data) + padder.finalize()
    enc = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
    return enc.update(padded) + enc.finalize()



# ВРАЗЛИВО: розшифрування ECB не може виявити підміну блоків.
def ecb_decrypt(ct: bytes, key: bytes) -> bytes:
    dec = Cipher(algorithms.AES(key), modes.ECB()).decryptor()
    padded = dec.update(ct) + dec.finalize()
    unpadder = padding.PKCS7(8 * BLOCK).unpadder()
    return unpadder.update(padded) + unpadder.finalize()



def ecb_encrypt_raw(data: bytes, key: bytes) -> bytes:
    """ECB без доповнення (довжина кратна 16) — для шифрування растрового зображення."""
    enc = Cipher(algorithms.AES(key), modes.ECB()).encryptor()
    return enc.update(data) + enc.finalize()



def blocks(data: bytes) -> list[bytes]:
    return [data[i:i + BLOCK] for i in range(0, len(data), BLOCK)]



def repeated_blocks(data: bytes) -> int:
    """Скільки блоків повторюють уже наявні (0 — усі блоки різні)."""
```

```
    counts = Counter(blocks(data))
    return sum(c - 1 for c in counts.values())



def record(user: str, role: str) -> bytes:
    """Запис із полями фіксованої довжини, вирівняними на межу блока."""
    return f"user={user}".ljust(BLOCK, ".").encode() + f"role={role}".ljust(BLOCK, ".").encode()



def cut_and_paste(victim_ct: bytes, donor_ct: bytes) -> bytes:
    """Замінити блок ролі шифротексту жертви блоком із чужого шифротексту."""
    return victim_ct[:BLOCK] + donor_ct[BLOCK:2 * BLOCK] + victim_ct[2 * BLOCK:]



def synthetic_image(size: int = 256) -> bytes:
    """Синтетичне зображення (відтінки сірого) з великими однотонними ділянками."""
    px = bytearray(size * size)
    for y in range(size):
        for x in range(size):
            v = 235                                              # світле тло
            if (x - 80) ** 2 + (y - 90) ** 2 < 55 ** 2:
                v = 40                                           # темне коло
            if 140 <= x < 230 and 40 <= y < 210:
                v = 120                                          # сірий прямокутник
            if 30 <= x < 230 and 225 <= y < 245:
                v = 0                                            # чорна смуга
            px[y * size + x] = v
    return bytes(px)
```

```
# crypto_lab/misuse_hash.py
"""Помилка 2 варіанта 14: хешування пароля без солі.

Без солі хеш — детермінована функція пароля: однакові паролі дають однакові хеші,
а заздалегідь обчислена таблиця «хеш → пароль» зламує всі слабкі паролі бази одним пошуком.
Виправлення — унікальна сіль на кожен пароль і повільна KDF (тут scrypt зі стандартної
бібліотеки, N = 2^14, r = 8, p = 1) з порівнянням за сталий час.
"""
from __future__ import annotations

import base64
import hashlib
```

```
import hmac
import os


SCRYPT_N, SCRYPT_R, SCRYPT_P = 2 ** 14, 8, 1
SALT_SIZE = 16



# ВРАЗЛИВО: швидкий хеш без солі — однакові паролі → однакові хеші.
def hash_unsalted(password: str) -> str:
    return hashlib.sha256(password.encode()).hexdigest()



def precompute_table(dictionary: list[str]) -> dict[str, str]:
    """Таблиця «хеш → пароль», яку порушник будує ОДИН раз — до отримання бази."""
    return {hash_unsalted(w): w for w in dictionary}



def crack_with_table(db: dict[str, str], table: dict[str, str]) -> dict[str, str]:
    """Зламати базу пошуком у готовій таблиці (без жодного обчислення хешу)."""
    return {user: table[h] for user, h in db.items() if h in table}



# БЕЗПЕЧНО: унікальна сіль + повільна KDF; формат scrypt$N$r$p$сіль$ключ.
def hash_salted(password: str) -> str:
    salt = os.urandom(SALT_SIZE)
    dk = hashlib.scrypt(password.encode(), salt=salt,
                        n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P)
    s, k = base64.b64encode(salt).decode(), base64.b64encode(dk).decode()
    return f"scrypt${SCRYPT_N}${SCRYPT_R}${SCRYPT_P}${s}${k}"



def verify_salted(stored: str, password: str) -> bool:
    _, n, r, p, salt_b64, dk_b64 = stored.split("$")
    salt, expected = base64.b64decode(salt_b64), base64.b64decode(dk_b64)
    dk = hashlib.scrypt(password.encode(), salt=salt, n=int(n), r=int(r), p=int(p))
    return hmac.compare_digest(dk, expected)
```

```
def crack_salted(stored: str, dictionary: list[str]) -> str | None:
    """Із сіллю таблиця марна: кожен запис доводиться перебирати окремо, з повільною KDF."""
    for w in dictionary:
        if verify_salted(stored, w):
            return w
    return None
```

```
# crypto_lab/sign_demo.py
"""Електронний підпис — ЛР4, варіант 14: RSA-PSS (2048 біт, SHA-256, MGF1-SHA256).


Запуск з каталогу lab04_crypto:  python -m crypto_lab.sign_demo
Ключ генерується щоразу заново й ніде не зберігається (ТЕСТОВИЙ КЛЮЧ — не використовувати поза стендом).
"""
from __future__ import annotations


import sys
import time


from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa


KEY_BITS = 2048
PUBLIC_EXPONENT = 65537
PSS = padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                  salt_length=padding.PSS.MAX_LENGTH)
HASH = hashes.SHA256()



def generate_keypair() -> rsa.RSAPrivateKey:
    # ТЕСТОВИЙ КЛЮЧ — не використовувати поза стендом
    return rsa.generate_private_key(public_exponent=PUBLIC_EXPONENT,
                                    key_size=KEY_BITS)



def sign(priv: rsa.RSAPrivateKey, message: bytes) -> bytes:
    return priv.sign(message, PSS, HASH)



def verify(pub: rsa.RSAPublicKey, signature: bytes, message: bytes) -> bool:
```

```
    """True — підпис дійсний; False — дані або підпис змінено."""
    try:
        pub.verify(signature, message, PSS, HASH)
        return True
    except InvalidSignature:
        return False



def max_salt_length(key_bits: int = KEY_BITS, hash_len: int = 32) -> int:
    """PSS.MAX_LENGTH = emLen − hLen − 2 (RFC 8017, п. 9.1.1), emLen = ⌈(modBits − 1) / 8⌉."""
    return (key_bits - 1 + 7) // 8 - hash_len - 2



def main() -> None:
    sys.stdout.reconfigure(encoding="utf-8")
    t = time.perf_counter()
    priv = generate_keypair()
    t_gen = time.perf_counter() - t
    pub = priv.public_key()
    msg = b"lab4 demo message: order #2026-14 approved"
    t = time.perf_counter()
    sig = sign(priv, msg)
    t_sign = time.perf_counter() - t
    t = time.perf_counter()
    ok = verify(pub, sig, msg)
    t_verify = time.perf_counter() - t

    print(f"Ключ RSA: {pub.key_size} біт, e = {pub.public_numbers().e}; генерація {t_gen * 1000:.0f} мс")
    print(f"Повідомлення: {msg!r}")
    print(f"Підпис: {len(sig)} Б, сіль PSS {max_salt_length()} Б; підпис {t_sign * 1000:.2f} мс, "
          f"перевірка {t_verify * 1000:.2f} мс")
    print(f"Підпис (перші 32 байти): {sig[:32].hex()}…")
    print("Перевірка коректного підпису:        ", ok)
    print("Повідомлення + b'x':                 ", verify(pub, sig, msg + b"x"))
    flipped_msg = bytes([msg[0] ^ 1]) + msg[1:]
    print("Змінено 1 біт повідомлення:          ", verify(pub, sig, flipped_msg))
    flipped_sig = sig[:-1] + bytes([sig[-1] ^ 1])
    print("Змінено 1 біт підпису:               ", verify(pub, flipped_sig, msg))
    sig2 = sign(priv, msg)
    print("Повторний підпис того самого тексту: ", "інший" if sig2 != sig else "той самий",
          "| дійсний:", verify(pub, sig2, msg))
    other = generate_keypair().public_key()
```

```
    print("Перевірка чужим відкритим ключем:    ", verify(other, sig, msg))
    pem = pub.public_bytes(serialization.Encoding.PEM, serialization.PublicFormat.SubjectPublicKeyInfo)
    print(f"Відкритий ключ (PEM, {len(pem)} Б): {pem.decode().splitlines()[1][:48]}…")



if __name__ == "__main__":
    main()
```

```
# crypto_lab/rsa_analysis.py
"""Аналіз навчального RSA: факторизація модуля, час генерації простих, кількість множень.

Факторизація потрібна, щоб показати, чому модуль із 28-бітовими множниками непридатний
для захисту: ρ-метод Полларда знаходить множник приблизно за √p кроків.
"""
from __future__ import annotations

import math
import random
import time

from crypto_lab.rsa_edu import fast_pow_mod, gen_prime



def trial_division(n: int) -> int:
    """Повний перебір непарних дільників до √n — O(√n) = O(p) кроків."""
    if n % 2 == 0:
        return 2
    d = 3
    while d * d <= n:
        if n % d == 0:
            return d
        d += 2
    return n



def pollard_rho(n: int, seed: int = 1) -> int:
    """ρ-метод Полларда (варіант Брента): нетривіальний дільник n за ≈ √p ітерацій."""
    if n % 2 == 0:
        return 2
    rng = random.Random(seed)
```

```
    while True:
        y, c, m = rng.randrange(1, n), rng.randrange(1, n), 128
        g = r = q = 1
        while g == 1:
            x = y
            for _ in range(r):
                y = (y * y + c) % n
            k = 0
            while k < r and g == 1:
                ys = y
                for _ in range(min(m, r - k)):
                    y = (y * y + c) % n
                    q = q * abs(x - y) % n
                g = math.gcd(q, n)
                k += m
            r *= 2
        if g == n:
            g = 1
            while g == 1:
                ys = (ys * ys + c) % n
                g = math.gcd(abs(x - ys), n)
        if g != n:
            return g



def time_it(fn, *args) -> tuple[object, float]:
    t = time.perf_counter()
    res = fn(*args)
    return res, time.perf_counter() - t



def multiplications(exp: int) -> tuple[int, int]:
    """(квадратів, множень на основу) у fast_pow_mod для показника exp."""
    _, steps = fast_pow_mod(3, exp, 1_000_003)
    return len(steps), bin(exp).count("1")



def factor_table(bits_list=(12, 16, 20, 24, 28, 32, 36), trial_up_to: int = 24):
    """Час генерації пари простих і факторизації n = p·q для заданої розрядності множників."""
    rows = []
    for bits in bits_list:
```

```
    (p, t_p) = time_it(gen_prime, bits)
    q = gen_prime(bits)
    while q == p:
        q = gen_prime(bits)
    n = p * q
    f_rho, t_rho = time_it(pollard_rho, n)
    assert f_rho in (p, q)
    t_trial = None
    if bits <= trial_up_to:
        f_tr, t_trial = time_it(trial_division, n)
        assert f_tr == min(p, q)
    rows.append({"bits": bits, "n_bits": n.bit_length(), "t_prime": t_p,
                 "t_trial": t_trial, "t_rho": t_rho})
return rows
```